

# JARVIS — Full Setup Guide

Turn a plain app into a voice assistant that reads your morning, talks back to you, and plugs into your own tools (calendar, inbox, dashboards). It's open source, so it stays yours — no subscription lock-in on the assistant itself.

**Time needed: ~20–30 minutes**

**Difficulty:** Beginner-friendly (you'll paste commands and let Claude do the heavy lifting)

💡 The whole idea: you don't build this by hand. You install **Claude Code**, hand it the project's GitHub link, and it pulls the files, installs the dependencies, and gets everything running for you. You just supply a few keys and connect your tools.

## ✓ Before you start

- ☐ A computer running **macOS 13+**, **Windows 10 (1809+)**, or **Linux (Ubuntu 20.04+ / Debian 10+)**
- ☐ **4 GB+ RAM** and an internet connection
- ☐ A **paid Claude plan** (Pro, Max, Team, or Enterprise) — Claude Code is **not** on the free Claude.ai plan
- ☐ A **GitHub account**
- ☐ An **ElevenLabs account** (the free tier gives ~10k voice credits/month to start)
- ☐ The **GitHub link** for the assistant project you want to install

You're a guest in Bubbly's Space. Get full access to Notion in your own workspace.

Set up workspace

## Step 1 — Install Claude Code

Claude Code is the agent that installs and runs everything. Open your terminal and run the command for your OS.

### macOS / Linux / WSL

```
curl -fsSL https://claude.ai/install.sh | bash
```

### Windows (PowerShell)

```
irm https://claude.ai/install.ps1 | iex
```

### Alternative (any OS, needs Node.js 18+)

```
npm install -g @anthropic-ai/claude-code
```

Then confirm it installed and log in:

```
claude --version # should print a version number
claude doctor    # checks your install and PATH
```

Close Click or Ctrl+\  
Resize Drag

## Set up workspace

```

claude --version    # should print a version number
claude doctor      # checks your install and PATH

```

The first time you launch it, it opens your browser to sign in with your Claude account:

claude

🐾 If you get `command not found: claude`, close and reopen your terminal (the installer updates your PATH). Still stuck? Run `claude doctor` — it tells you exactly what's wrong.

## Step 2 — Hand Claude the project and let it install itself

Make a folder for the assistant, open Claude Code inside it, then give it the repo.

```
mkdir jarvis && cd jarvis
claude
```

Now paste this prompt into Claude Code (swap in your real link):

Clone this repo into the current folder, read its README, install all dependencies, and get it running. Tell me anything you need from me (API keys, config, etc.) as you go:

You're a guest in Bubbly's Space. Get full access to Notion in your own workspace.

Set up workspace

Now paste this prompt into Claude Code (swap in your real link):

Clone this repo into the current folder, read its README, install all dependencies, and get it running. Tell me anything you need from me (API keys, config, etc.) as you go:

<PASTE-YOUR-REPO-URL>

Claude will pull the files, set up the dependencies, and start the project on its own. When it needs something from you (like a key), it'll pause and ask — just answer in plain English.

💡 If you don't have a project picked yet, search GitHub for open-source "Claude voice assistant" or "voice MCP" projects and drop that link in. The setup flow below works the same way.

## Step 3 — Add your ElevenLabs voice

This is what makes it *talk*.

### 3a. Get your API key

1. Sign in at [elevenlabs.io](https://elevenlabs.io)
2. Open your profile → API Keys → create a new key
3. Copy it somewhere safe

### 3b. Pick a voice

1. Go to the **Voices / Voice Library** section
2. Preview a few and choose one you like
3. Copy its **Voice ID** (you'll paste this into the config)

### 3c. Plug it in

?

You're a guest in Bubbly's Space. Get full access to Notion in your own workspace.

Set up workspace

✕

### 3c. Plug it in

Easiest path — let the official ElevenLabs MCP server handle speech. First install `uv` (a small helper), then add the server to Claude Code:

```
# install uv (macOS/Linux)
curl -LsSf https://astral.sh/uv/install.sh | sh

# add ElevenLabs to Claude Code
claude mcp add elevenlabs --env ELEVENLABS_API_KEY=<your-key-here> -- uvx elevenlabs-mcp
```

Or, if your project uses a `.mcp.json` file, add this block and paste your key inside:

```
{
  "mcpServers": {
    "elevenlabs": {
      "command": "uvx",
      "args": ["elevenlabs-mcp"],
      "env": {
        "ELEVENLABS_API_KEY": "<your-key-here>",
        "ELEVENLABS_VOICE_ID": "<your-voice-id>"
      }
    }
  }
}
```

?

You're a guest in Bubbly's Space. Get full access to Notion in your own workspace.

Set up workspace

✕

Then just tell Claude:

|

Use ElevenLabs for the voice output, with the voice ID I set. Read this line out loud to test it:  
"Good morning — your assistant is online."

Restart Claude Code if it doesn't pick up the new server right away.

|

⚠ ElevenLabs voice generation uses credits. The free tier is plenty for testing; heavy daily use may need a paid ElevenLabs plan.

## Step 4 — Connect your tools

Now link the assistant to the things you actually use — calendar, inbox, dashboards, notes, whatever. Each tool connects as an MCP server, exactly like ElevenLabs did.

The pattern is always the same:

```
claude mcp add <tool-name> --env KEY=value -- <command-to-run-that-server>
```

To see what's connected at any time:

```
claude mcp list
```

Then let Claude wire the tool into your assistant's routine:

## Set up workspace

I've connected my calendar and email. Update the assistant so my morning brief pulls from both — show me what's on my calendar today and which emails still need a reply.

💡 Most popular tools (Google Calendar, Gmail, Notion, Slack, etc.) have a ready-made MCP server. Search "[tool name] MCP server" to find the exact install command, then add it with the same `claude mcp add` pattern above. Start with one or two tools — you can always add more later.

Ask for it:

Give me my morning brief out loud: what's pending, what's important, and which emails I still haven't replied to.

Instead of scrolling through 40 emails, you get a short spoken summary of what actually needs you. Once you like the format, ask Claude to save it as a reusable command or a scheduled routine so you can trigger it every morning with one word.

Because the project is open source, the assistant stays yours forever:

- The code lives on your machine — you own it and can change anything
- No company can flip a switch and start charging you for the assistant later
- You only pay for what you actually use (your Claude plan, and ElevenLabs credits if you go



⚡ Command cheat sheet

```
# install Claude Code (macOS/Linux)
curl -fsSL https://claude.ai/install.sh | bash

# verify
claude --version
claude doctor

# start Claude Code in a folder
cd jarvis && claude

# add a tool (MCP server)
claude mcp add <name> --env KEY=value -- <command>

# list connected tools
claude mcp list
```